

(12) **United States Patent**
Hubbard

(10) **Patent No.:** **US 12,405,749 B2**
(45) **Date of Patent:** ***Sep. 2, 2025**

(54) **SPECIFYING MEDIA TYPE IN WRITE COMMANDS**

(71) Applicant: **Micron Technology, Inc.**, Boise, ID (US)

(72) Inventor: **Daniel J. Hubbard**, Boise, ID (US)

(73) Assignee: **Micron Technology, Inc.**, Boise, ID (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/368,404**

(22) Filed: **Sep. 14, 2023**

(65) **Prior Publication Data**
US 2024/0004587 A1 Jan. 4, 2024

Related U.S. Application Data

(63) Continuation of application No. 15/937,146, filed on Mar. 27, 2018, now Pat. No. 11,803,325.

(51) **Int. Cl.**
G06F 3/06 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 3/0659** (2013.01); **G06F 3/0611** (2013.01); **G06F 3/0632** (2013.01); **G06F 3/067** (2013.01)

(58) **Field of Classification Search**
CPC G06F 3/0659; G06F 3/0611; G06F 3/0632; G06F 3/067; G06F 3/061; G06F 3/0649; G06F 3/0685

(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,948,118 A 9/1999 Higurashi
6,119,259 A 9/2000 Jeong
(Continued)

FOREIGN PATENT DOCUMENTS

CN 1746856 A 3/2006
CN 106354425 A 1/2017
(Continued)

OTHER PUBLICATIONS

PCT Notification Of Transmittal Of The International Search Report And The Written Opinion Of The International Searching Authority, for the PCT Application No. PCT/US2019/024265, mailed Jul. 11, 2019, 9 pages.

(Continued)

Primary Examiner — Hosain T Alam
Assistant Examiner — Zubair Ahmed
(74) *Attorney, Agent, or Firm* — Lowenstein Sandler LLP

(57) **ABSTRACT**

Systems and methods for specifying storage media types in write commands executable by storage devices are disclosed. An example system comprises: a plurality of memory devices and a controller operatively coupled to the memory devices, the controller configured to: receive a write command specifying a data item and an identifier of a data stream comprising the data item; determine, by parsing the identifier of the data stream, a data stream attribute shared by data items comprised by the data stream; identify, based on the data stream attribute, a memory device managed by the controller; and transmit, to the memory device, an instruction specifying the data item.

17 Claims, 8 Drawing Sheets

Mapping table
400

410A	Attribute value 420A	Memory device ID 430A
410B	Attribute value 420B	Memory device ID 430B
410N	Attribute value 420N	Memory device ID 430N

(58) **Field of Classification Search**

USPC 711/154, 100
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

6,536,011	B1	3/2003	Jang	
8,396,839	B1	3/2013	Huang	
9,128,820	B1	9/2015	Malina	
9,183,218	B1	11/2015	Wallace	
2002/0026564	A1	2/2002	Kusakabe	
2006/0077883	A1	4/2006	Ando	
2007/0245103	A1	10/2007	Lam	
2007/0258650	A1	11/2007	Marchal	
2008/0294980	A1	11/2008	Thienot	
2008/0303813	A1	12/2008	Joung	
2009/0012934	A1	1/2009	Yerigan	
2010/0199053	A1	8/2010	Otani	
2010/0214303	A1	8/2010	Kaneko	
2011/0035548	A1	2/2011	Kimmel	
2011/0310728	A1	12/2011	Wu	
2013/0103891	A1	4/2013	Sharon	
2013/0159626	A1	6/2013	Katz	
2014/0059406	A1	2/2014	Hyun	
2014/0075240	A1	3/2014	Maeda	
2015/0271857	A1	9/2015	Ribeiro	
2015/0279462	A1	10/2015	Iyer	
2015/0339319	A1	11/2015	Malina	
2016/0054931	A1	2/2016	Romanovsky	
2016/0117266	A1	4/2016	Anderson	
2016/0147454	A1	5/2016	Meir	
2016/0179386	A1	6/2016	Zhang	
2016/0266792	A1	9/2016	Amaki et al.	
2016/0283116	A1	9/2016	Ramalingam	
2017/0017411	A1	1/2017	Choi	
2017/0228157	A1	8/2017	Yang	
2017/0242625	A1 *	8/2017	Pandurangan	G06F 3/0659
2017/0285948	A1 *	10/2017	Thomas	G06F 3/0616
2018/0024737	A1	1/2018	Dubeyko	
2019/0004736	A1 *	1/2019	Lee	G06F 3/0679
2020/0045532	A1	2/2020	Deaconu	

FOREIGN PATENT DOCUMENTS

CN	107274331	A	10/2017
EP	2757479	A1	7/2014
EP	3118745	A1	1/2017
KR	1020130135618	A	12/2013
KR	1020150020136	A	2/2015
KR	1020170008153	A	1/2017
TW	200928732	A	7/2009
TW	I371691	A	9/2012

OTHER PUBLICATIONS

PCT Notification of Concerning Transmittal of International Preliminary Report on Patentability for PCT Application No. PCT/US2019/024265, mailed Oct. 8, 2020, 9 pages.
Working Draft American National Standard, Project T10/BSR INCITS 506, Information technology—SCSI Block Commands—4 (SBC-4), 448 pages, Aug. 17, 2017.

JVM Express 1.3, NVM Express Revision 1.3, May 1, 2017, Apr. 26, 2017, 282 pages.

USPTO, Office Action for U.S. Appl. No. 15/937,146, mailed Feb. 11, 2019.

USPTO, Final Office Action for U.S. Appl. No. 15/937,146, mailed Jul. 1, 2019.

USPTO, Office Action for U.S. Appl. No. 15/937,146, mailed Jan. 28, 2020.

USPTO, Advisory Action for U.S. Appl. No. 15/937,146, mailed Oct. 9, 2019.

USPTO, Final Office Action for U.S. Appl. No. 15/937,146, mailed May 27, 2020.

USPTO, Advisory Action for U.S. Appl. No. 15/937,146, mailed Jul. 31, 2020.

USPTO, Office Action for U.S. Appl. No. 15/937,146, mailed Mar. 8, 2021.

USPTO, Final Office Action for U.S. Appl. No. 15/937,146, mailed Jun. 25, 2021.

USPTO, Notice of Allowance for U.S. Appl. No. 15/937,146, mailed May 11, 2023.

USPTO, Notice of Allowance for U.S. Appl. No. 15/937,146, mailed Jun. 23, 2023.

PCT Notification Of Transmittal Of The International Search Report And The Written Opinion Of The International Searching Authority, for the PCT Application No. PCT/US2018/065813, mailed Apr. 22, 2019, 13 pages.

PCT Notification Concerning Transmittal Of International Preliminary Report On Patentability (Chapter I Of The Patent Cooperation Treaty) for PCT Application No. PCT/US2018/065813, mailed Jul. 2, 2020, 8 pages.

U.S. Appl. No. 15/937,146, filed Mar. 27, 2018.

USPTO, Office Action for U.S. Appl. No. 15/849,014, mailed May 6, 2019.

USPTO, Final Office Action for U.S. Appl. No. 15/849,014, mailed Nov. 4, 2019.

USPTO, Office Action for U.S. Appl. No. 15/849,014, mailed Feb. 6, 2020.

USPTO, Advisory Action for U.S. Appl. No. 15/849,014, mailed Jan. 27, 2020.

USPTO, Notice of Allowance for U.S. Appl. No. 15/849,014, mailed Aug. 27, 2020.

U.S. Appl. No. 15/849,014, filed Dec. 20, 2017.

USPTO, Office Action for U.S. Appl. No. 17/100,148, mailed Jun. 18, 2021.

USPTO, Notice of Allowance for U.S. Appl. No. 17/100,148, mailed Nov. 4, 2021.

USPTO, Corrected Notice of Allowability for U.S. Appl. No. 17/100,148, mailed Feb. 17, 2022.

KR Notice of Preliminary Rejection for Korean Patent Application No. 10-2020-7020753, issued Sep. 1, 2021.

The extended European Search Report for EP Application No. 18893290.9, mailed on Sep. 23, 2021, 11 pages.

The extended European Search Report for EP Application No. 19778234.5, mailed on Nov. 10, 2021, 11 pages.

Taiwan Office Action for Application No. 107145348, issued Dec. 20, 2021, 11 pages.

Taiwan office action for Application No. 111126674 with a search report, issued on Sep. 16, 2022, 12 pages.

USPTO, Notice of Allowance for U.S. Appl. No. 17/675,945, mailed on Dec. 21, 2022.

* cited by examiner

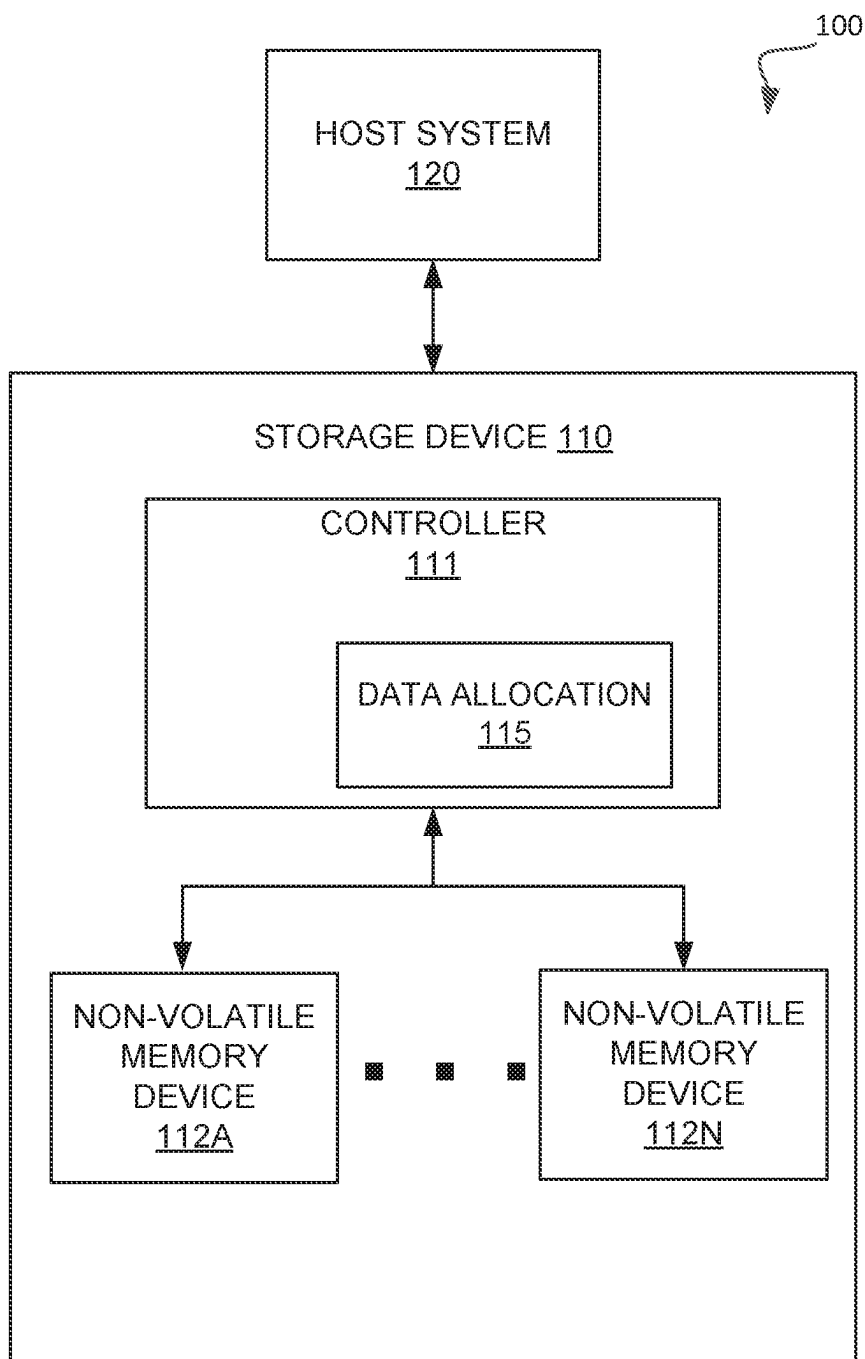


FIG. 1

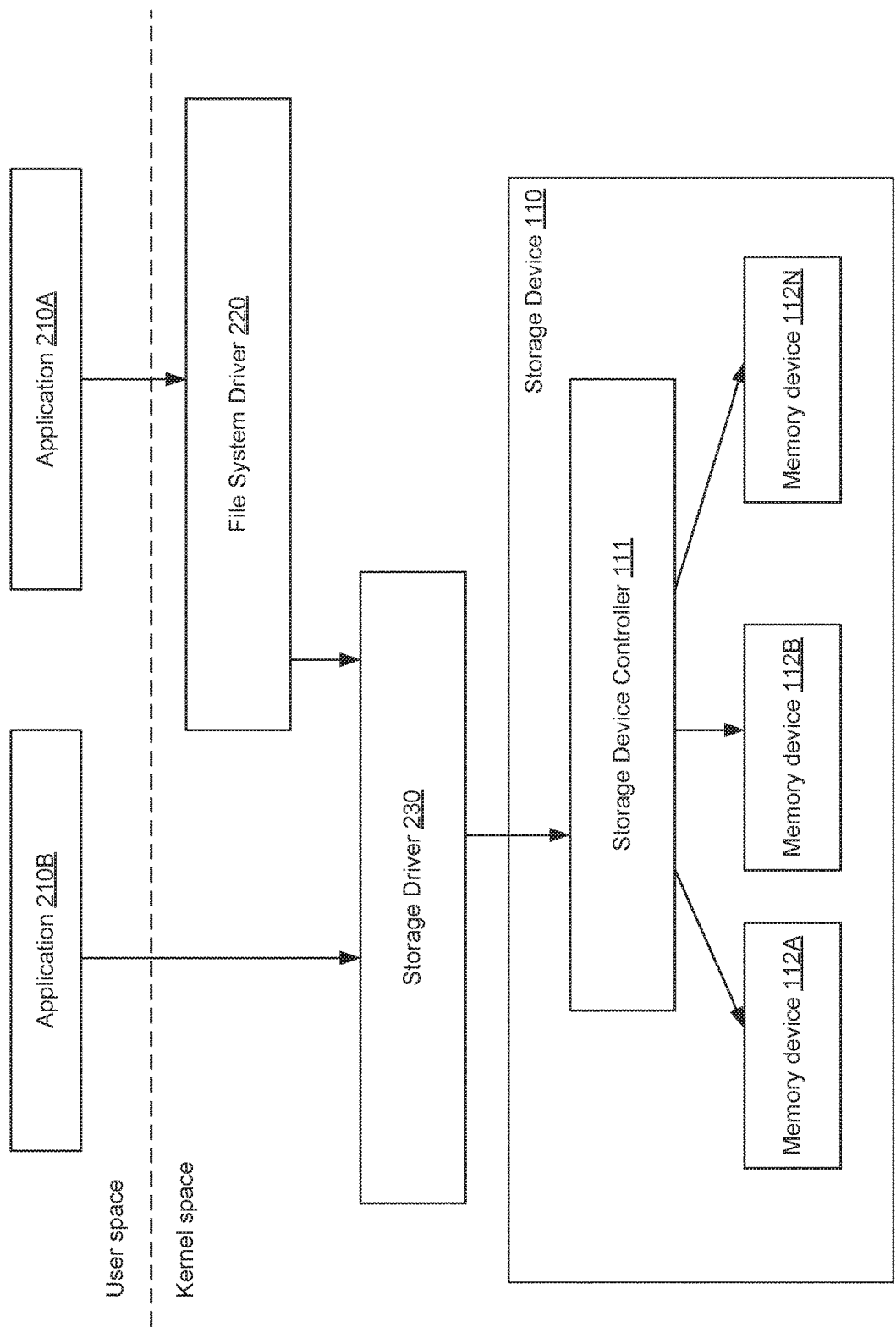


FIG. 2

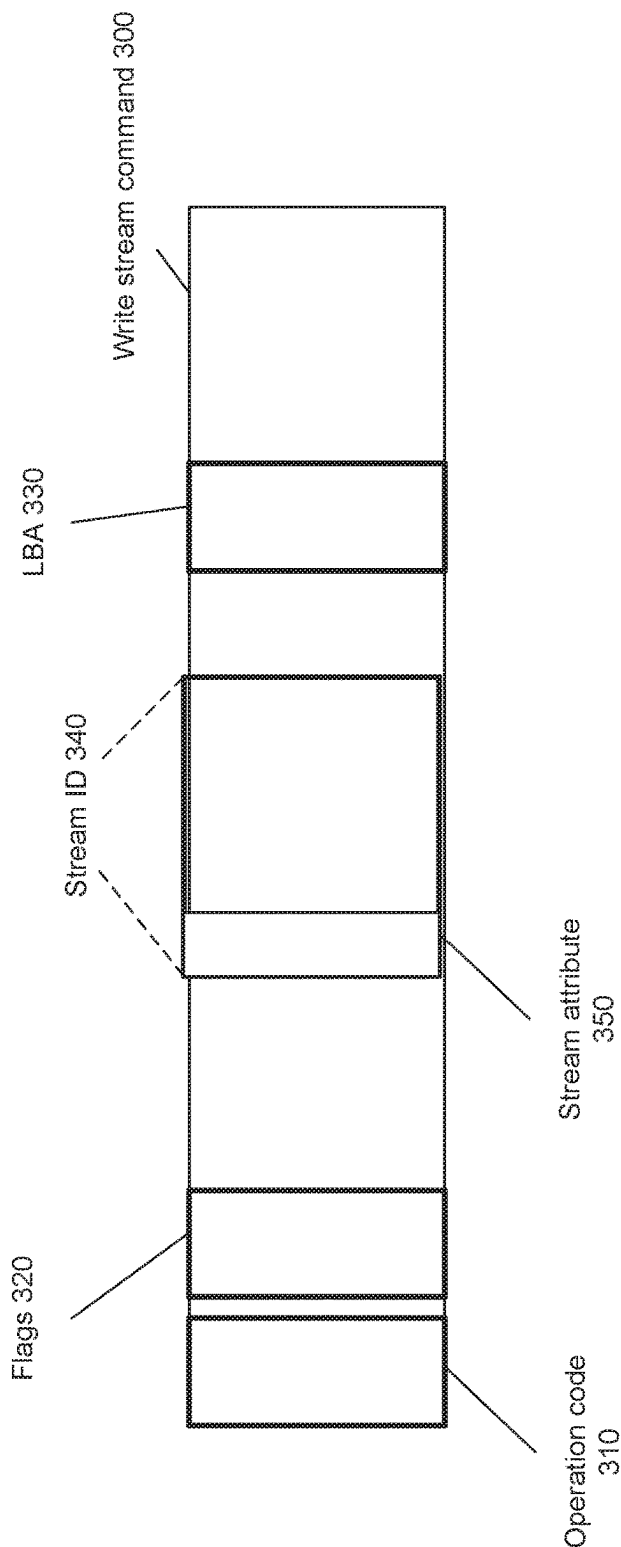


FIG. 3

Mapping table
400

410A	Attribute value <u>420A</u>	Memory device ID <u>430A</u>
410B	Attribute value <u>420B</u>	Memory device ID <u>430B</u>
410N	Attribute value <u>420N</u>	Memory device ID <u>430N</u>

FIG. 4

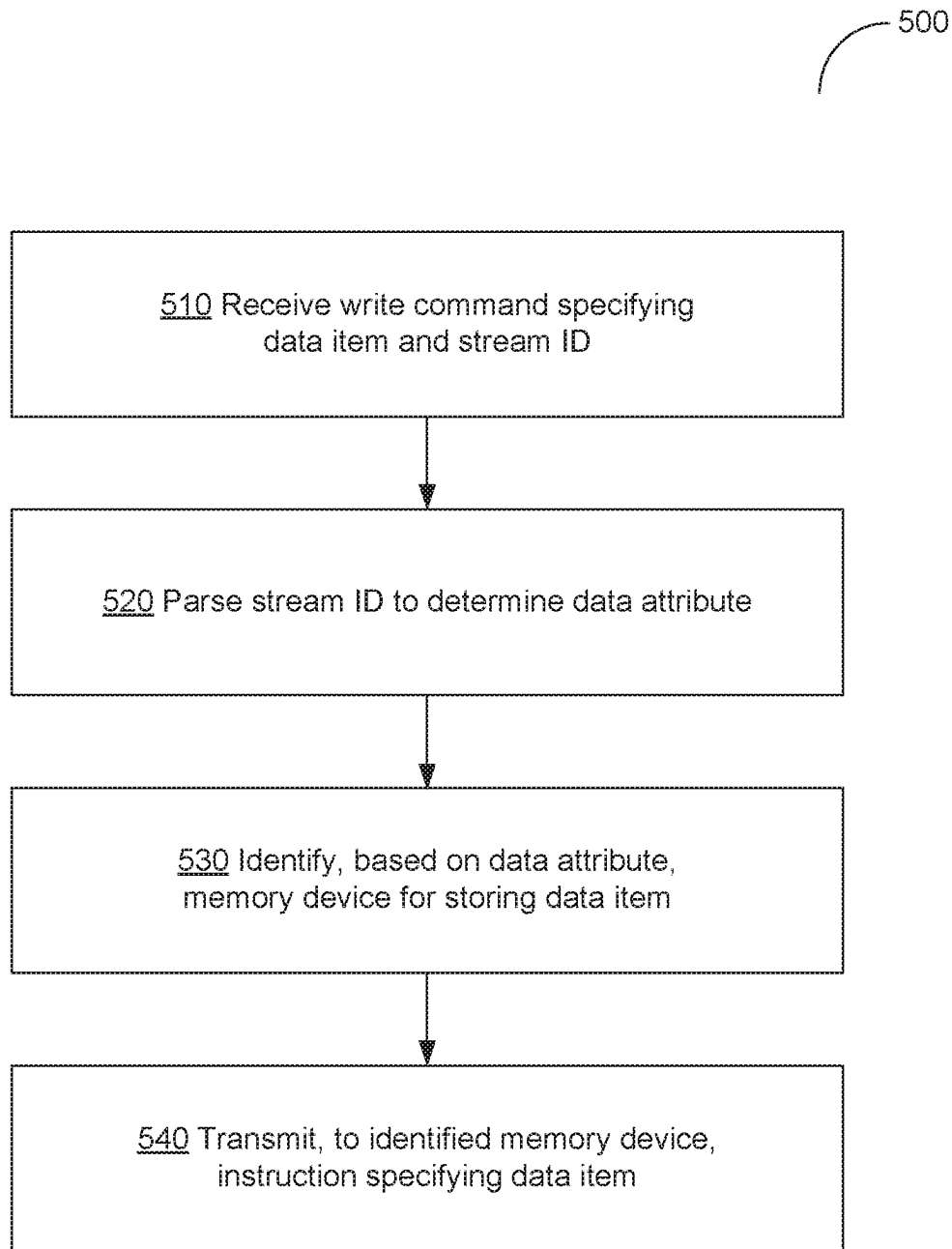


FIG. 5

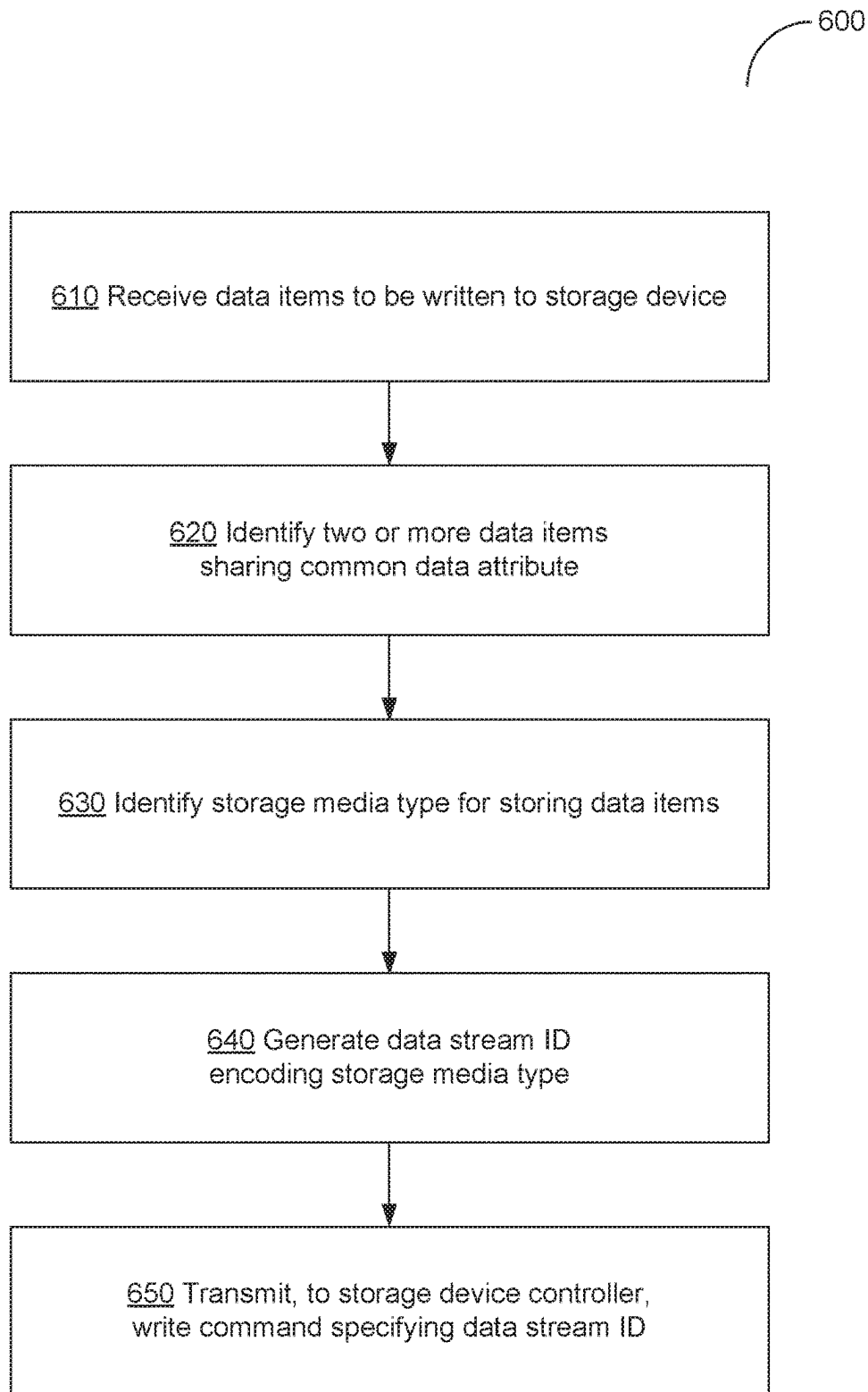


FIG. 6

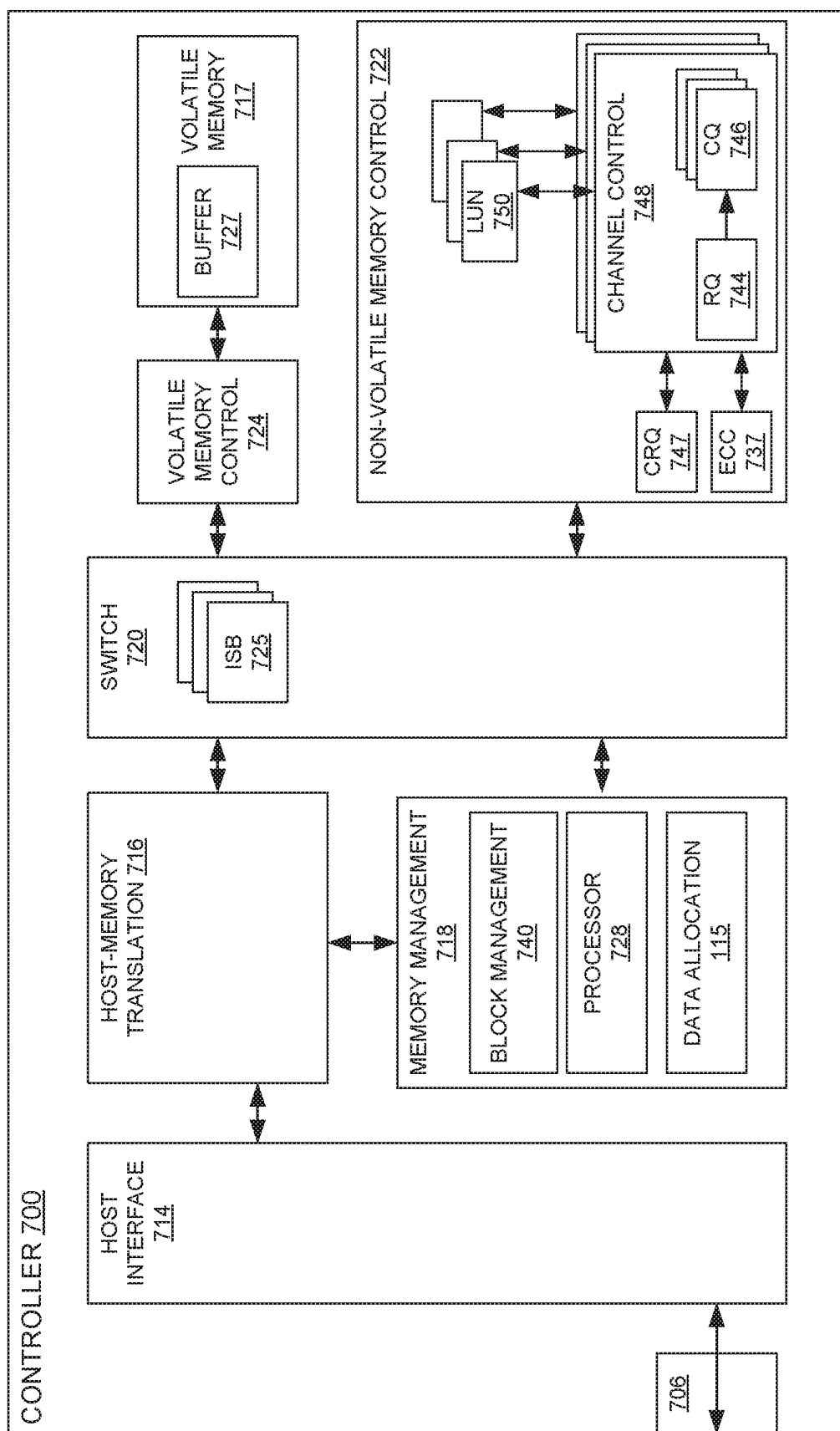


FIG. 7

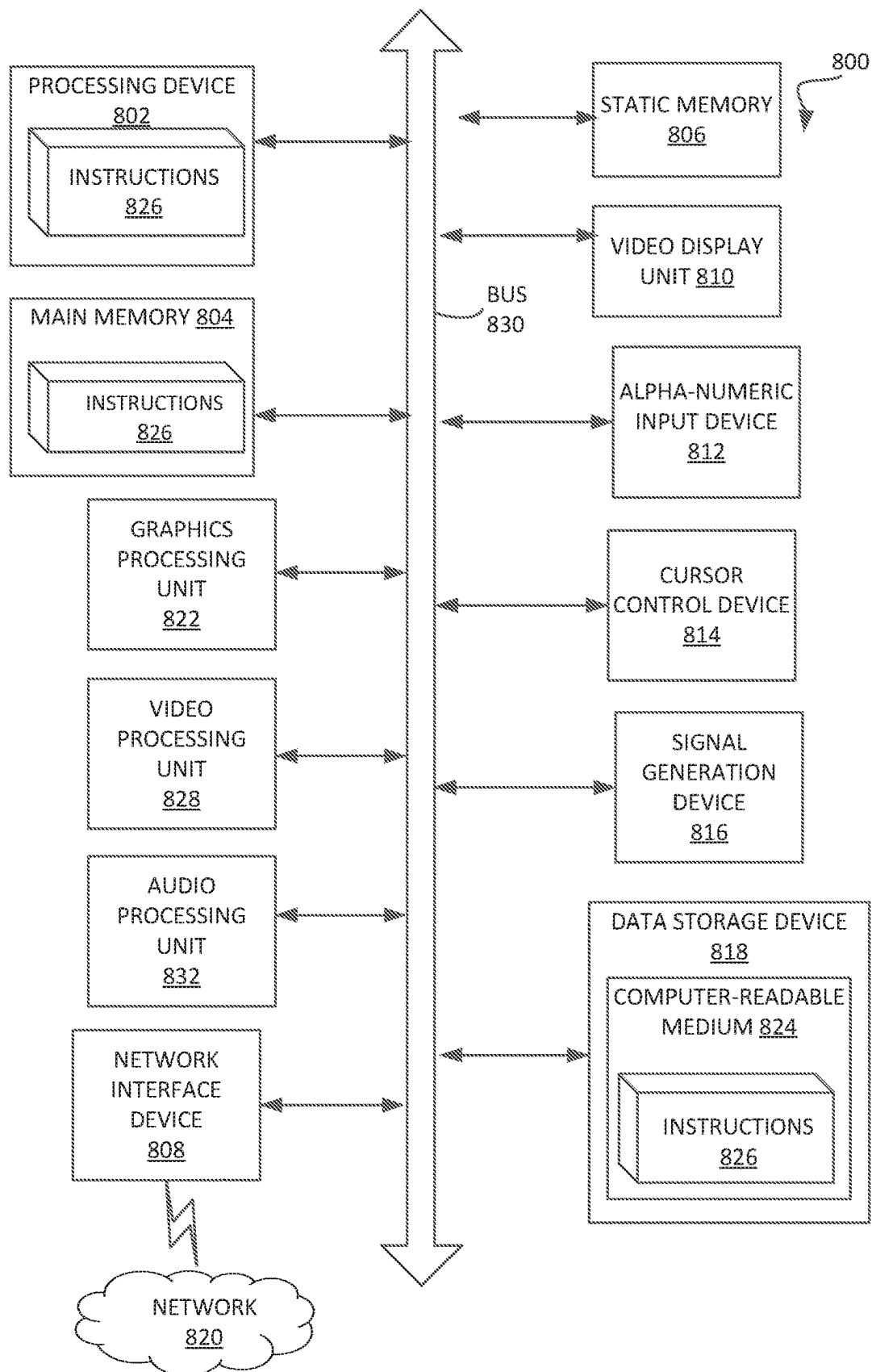


FIG. 8

1

SPECIFYING MEDIA TYPE IN WRITE COMMANDS

RELATED APPLICATIONS

This application is a continuation of U.S. patent application Ser. No. 15/937,146 filed on Mar. 27, 2018, the entire content of which is incorporated by reference herein.

TECHNICAL FIELD

The present disclosure generally relates to memory systems, and more specifically, relates to specifying media types in write commands executable by memory systems.

BACKGROUND

A memory system can be a storage device, such as a solid-state drive (SSD), and can include one or more memory components that store data. For example, a memory system can include memory devices such as non-volatile memory devices and volatile memory devices. In general, a host system can utilize a memory system to store data at the memory devices of the memory system and to retrieve data stored at the memory system.

BRIEF DESCRIPTION OF THE DRAWINGS

The present disclosure will be understood more fully from the detailed description given below and from the accompanying drawings of various implementations of the disclosure.

FIG. 1 schematically illustrates an example computing environment operating in accordance with one or more aspects of the present disclosure;

FIG. 2 schematically illustrates a programming model which may be implemented by the host system in communication with the storage device controller managing one or more storage devices, in accordance with one or more aspects of the present disclosure;

FIG. 3 schematically illustrates an example structure of the write stream command, in accordance with one or more aspects of the present disclosure;

FIG. 4 schematically illustrates an example data placement structure mapping data attribute values to the storage media types and/or memory device identifiers, in accordance with one or more aspects of the present disclosure;

FIG. 5 is a flow diagram of an example method of identifying memory devices based on data stream attributes, in accordance with one or more aspects of the present disclosure;

FIG. 6 is a flow diagram of an example method of specifying data stream attributes identifying storage media type to be utilized for storing the data items, in accordance with one or more aspects of the present disclosure;

FIG. 7 is a block diagram of an example storage device controller operating in accordance with one or more aspects of the present disclosure; and

FIG. 8 schematically illustrates a block diagram of an example computer system in which implementations of the present disclosure may operate.

DETAILED DESCRIPTION

Aspects of the present disclosure are directed to specifying media types in write commands that are transmitted by

2

a host system to a memory system and are further directed to executing such write commands by the memory system.

An example of a memory system is a storage system (hereinafter referred to as a storage device), such as a solid-state drive (SSD). In some embodiments, the memory system is a hybrid memory/storage system. A storage device is used as an example of a memory system throughout this document. A storage device can multiple memory devices having various storage media types, including negative-and (NAND) flash memory (utilizing single-level cell (SLC), triple-level cell (TLC), and/or quad-level cell (QLC) blocks), dynamic random access memory (DRAM), static random access memory (SRAM), and/or 3D cross point memory. Systems and methods described herein identify data allocation to memory devices that optimizes performance, endurance, and/or other operational aspects of the memory devices.

Data items to be written to the storage device may be grouped, by the host system, into one or more data streams, such that each data stream would contain data items belonging to the same group of data items (e.g., the data items associated with a single data structure, such as a file or a database). Thus, the data items contained by a single data stream may share one or more attributes, including attributes that reflect anticipated media usage patterns. Examples of such attributes include the data retention time (also referred to as the “stream temperature”) and the workload type.

Accordingly, responsive to receiving a write command specifying one or more data items to be written to a memory device and one or more attributes that reflect anticipated media usage patterns, the storage device controller may analyze the data attributes in order to identify the storage media type that is best suited for storing the data items, and may then identify an available memory device supporting the identified storage media type. In an illustrative example, the data allocation strategy implemented by the storage device controller may optimize the device performance, e.g., by placing data items with a low anticipated retention time on DRAM or SRAM memory devices, which demonstrate higher performance and lower latency than NAND memory devices. In another illustrative example, the storage device controller may determine the data allocation for optimizing device endurance, e.g., by placing data items with a low anticipated retention time on DRAM or SRAM memory devices, which, unlike NAND memory devices, are not susceptible to wear caused by multiple write cycles.

Alternatively, the storage media type that is best suited for storing the data items may be determined by the host, which may directly specify the storage media type by data attributes that are communicated to the storage device controller. Similarly to the above-described examples of data allocation performed by the storage device controller, the host may determine the data allocation for optimizing device performance, endurance, and/or other operational aspects of the memory devices. Responsive to receiving a write command specifying a data attribute identifying a storage media type, the storage device controller may identify an available memory device supporting the identified storage media type, and may issue an instruction causing the identified memory device to store the specified data items.

As noted herein above, the host may group the data to be written to the storage device into several data streams. A data stream may be identified by a dedicated data stream identifier field of each write command transmitted by the host system to the storage device controller. In certain implementations, one or more bits of the stream identifier field of the write command may be utilized for specifying one or

more data attributes shared by the data items of the data stream. Thus, each write command transmitted by the host system to the storage device controller may not only identify the stream, but also indicate the data attributes which are shared by the data items of the data stream. The storage device controller may utilize the stream identifying information enhanced by the data attributes in order to determine the data allocation to available memory devices that would optimize performance, endurance, and/or other operational aspects of the memory devices.

Thus, aspects of the present disclosure represent significant improvements over various common implementations of storage devices and systems, by enhancing storage write commands with data stream attributes reflecting expected media usage patterns or directly specifying the storage media type, in order to optimize the usage of the storage media. Various aspects of the above referenced methods and systems are described in details herein below by way of examples, rather than by way of limitation.

FIG. 1 schematically illustrates an example computing environment 100 operating in accordance with one or more aspects of the present disclosure. In general, the computing environment 100 may include a host system 120 that uses the storage device 110. For example, the host system 120 may write data to the storage device 110 and read data from the storage device 110. The host system 120 may be a computing device such as a desktop computer, laptop computer, network server, mobile device, or such computing device that includes a memory and a processing device. The host system 120 may include or be coupled to the storage device 110 so that the host system 120 may read data from or write data to the storage device 110. For example, the host system 120 may be coupled to the storage device 110 via a physical host interface. Examples of a physical host interface include, but are not limited to, a serial advanced technology attachment (SATA) interface, a peripheral component interconnect express (PCIe) interface, universal serial bus (USB) interface, an NVMe Express (NVMe), Fibre Channel, Serial Attached SCSI (SAS), etc. The physical host interface may be used to transmit data between the host system 120 and the storage device 110. In an illustrative example, the host system 120 may be represented by the computer system 800 of FIG. 8.

As shown in FIG. 1, the storage device 110 may include a controller 111 and storage media, such as memory devices 112A to 112N. In an illustrative example, the memory devices 112A to 112N may include non-volatile memory devices, such as NAND flash memory. Each NAND flash memory device may include one or more arrays of memory cells such as single level cells (SLCs), multi-level cells (MLCs), or quad-level cells (QLCs). Each of the memory cells may store bits of data (e.g., data blocks) used by the host system 120. The memory cells of the memory devices 112A to 112N may be grouped as memory pages or data blocks that may refer to a unit of the memory device used to store data.

In another illustrative example, the memory devices 112A to 112N may include one or more cross-point arrays of non-volatile memory cells. The cross-point array can be a 3D cross point array. Cross point array memory devices implement a cross-point architecture that positions selectors and memory cells at intersections of perpendicular wires. Each cell stores a single bit encoded by either a high- or low-resistance state of the cell. To improve storage density, the 3D cross point cells are stackable in three dimensions. A cross-point array of non-volatile memory can perform bit storage based on a change of bulk resistance, in conjunction

with a stackable cross-gridded data access array. Additionally, in contrast to many Flash-based memory, cross point non-volatile memory can perform a write in-place operation, where a non-volatile memory cell can be programmed without the non-volatile memory cell being previously erased.

In yet another illustrative example, the memory devices 112A to 112N may include one or more volatile memory devices, such as static random access memory (SRAM), dynamic random access memory (DRAM), and/or synchronous dynamic random access memory (SDRAM) devices.

The controller 111 may communicate with the memory devices 112A to 112N to perform operations including reading data from or writing data to the memory devices 112A-112N. The controller 111 may include hardware such as one or more integrated circuits, firmware, or a combination thereof. In operation, the controller 111 may receive commands from the host system 120 and may convert the commands into instructions to perform the desired actions with respect to the memory devices 112A-112N. In various illustrative examples, the controller 111 may implement other operations such as wear leveling, garbage collection, error detection and error-correcting code (ECC), encryption, caching, and/or address translations between a logical block address and a physical block address that are associated with the memory devices 112A-112N. In certain implementations, the controller 111 may be responsible for moving the data that is stored on the volatile memory devices to non-volatile memory devices (e.g., responsive to detecting a power failure or other pre-defined event), in order to provide persistent storage of all data written to the storage device 110. Responsive to detecting a symmetric pre-defined event (e.g., the storage device power-up), the controller 111 may move the data back to non-volatile memory devices.

In order to implement the systems and methods of the present disclosure, the controller 111 may include a data allocation functional component 115 that may be employed to allocate the incoming data to memory devices 112A-112N. It should be noted that the component designation is of a purely functional nature, i.e., the functions of the data allocation component may be implemented by one or more hardware components and/or firmware modules of the controller 111, as described in more detail herein below. Furthermore, the storage device 110 may include additional circuitry or components that are omitted from FIG. 1 for clarity and conciseness.

FIG. 2 schematically illustrates a programming model which may be implemented by the host system 120 in communication with the storage device controller 111 managing one or more memory devices 112A-112N, in accordance with one or more aspects of the present disclosure. As schematically illustrated by FIG. 2, the host system may execute one or more applications 210A-210B. In an illustrative example, the application 210A may be in communication with the file system driver 220, which may be running in the kernel space of the host system 120 and may be employed for processing certain system calls, such as read and write calls initiated by one or more applications 210, including the application 210A, running in the user space of the host system 120. The file system driver 220 may be employed to translate the read, write, and other system calls issued by the application 210A into low-level application programming interface (API) calls to the storage driver 230, which, in turn may communicate to the device controller 111 controlling one or more memory devices 112A-112N. The storage driver 230 may be running in the kernel mode of the host system and may be employed to process API calls

5

issued by the file system driver **220** and/or system calls issued by the application **210B** into storage interface commands to be processed by the storage device controller **111** managing one or more memory devices **112A-112N**.

In an illustrative example, the storage driver **230** may implement a block storage model, in which the data is grouped into blocks of one or more pre-defined sizes and is addressable by a block number. The block storage model may implement “read” and “write” command for storing and retrieving blocks of data. In another illustrative example, the storage driver **230** may implement a key-value storage model, in which the data is represented by the “value” component of a key-value pair is addressable by the “key” component of the key-value pair. The key value storage model may implement “put” and “get” commands, which are functionally similar to the “write” and “read” commands of the block storage model. Thus, the term “data item” as used herein may refer to a data block or to a key-value pair.

The application **210A-210B** and/or the storage driver **230** executed by the host system **120** may group, into several data streams, the data to be written to the memory devices **112**, such that the data items belonging to the same data stream would share one or more data attributes. In an illustrative example, a data attribute may reflect the anticipated retention time of the data stream (also referred to as the “stream temperature”), such that a “hot” data stream would comprise short-living data items which are likely to be overwritten within a relatively short period of time (e.g., a period of time falling below a pre-defined low threshold), while a “cold” data stream comprise static data items which are not likely to be overwritten for a relatively long period of time (e.g., a period of time exceeding a pre-defined high threshold). In an illustrative example, the data stream temperature may be communicated to the storage driver **230** by the application **210** which produces the data stream and thus is presumably aware of its anticipated retention time. The data stream temperature may be communicated to the storage driver **230**, e.g., via an Input/Output Control (IOCTL) system call. Alternatively, the data stream temperature may be determined by the storage driver **230**, which may buffer the incoming data to be written to the memory devices **112A-112N**, and may estimate the anticipated retention time based on the average frequency of overwrite operations requested by the application **210** with respect to one or more data items to be written to the memory devices **112A-112N**. The storage driver **230** may then group the buffered data to be written to the storage device **110** into two or more data streams, and may issue stream write commands indicating the data stream temperature to the storage device controller **111**, as described in more detail herein below.

In another illustrative example, a data attribute may reflect the workload type of the data stream, e.g., the “log data” attribute indicating that the data represents the logging data related to one or more databases and/or file systems or “user data” attribute indicating that the data represents other (not related to database or file system logs) types of data. The data stream workload type may be communicated to the storage driver **230** by the application **210** which produces the data stream and thus is presumably aware of its workload type. The data stream workload type may be communicated to the storage driver **230**, e.g., via an Input/Output Control (IOCTL) system call. The storage driver **230** may group the data into one or more data streams based on the workload type attribute, and may issue stream write commands indicating the workload type to the storage device controller **111**, as described in more detail herein below.

6

In another illustrative example, the storage driver **230** may determine, based on the workload type and/or anticipated retention time of the data stream, a storage media type which is best suited for storing the data items comprised by the data stream. In one embodiment, the storage driver **230** may interpret the workload type attribute as indicative of the anticipated retention time, e.g., by assuming that data items associated with the “log data” attribute would have longer retention time than data items that are marked as the “user data.” The storage driver **230** may group the data into one or more data streams based on the anticipated retention time and/or workload type attributes, and may issue stream write commands indicating the storage media type to the storage device controller **111**, as described in more detail herein below.

In certain implementations, the data stream may be identified by a dedicated field in each write command transmitted by the host system to the storage device controller. FIG. **3** schematically illustrates an example structure of the write stream command, in accordance with one or more aspects of the present disclosure. The write stream command **300** may include, among other fields, the operation code field **310** specifying the command type (e.g., the write stream command). The write stream command **300** may further include the flags field **320** specifying one or more parameters of the command. The write stream command **300** may further include the logical block address (LBA) field **330** specifying the LBA of the data being stored on the storage device. The write stream command **300** may further include the stream identifier field **340** represented by a bit string, which may be interpreted as an unsigned integer value. One or more bits (such as a group of one or more most significant bits or a group of or more least significant bits) of the stream identifier field **340** may be utilized for specifying one or more data stream attributes **350** shared by the data items of the data stream. In an illustrative example, one or more bits of the stream identifier field **340** may be utilized for specifying the data stream temperature (e.g., “0” indicating a cold stream and “1” indicating a hot stream, or “00” indicating unknown stream temperature, “01” indicating a cold stream, “10” indicating medium stream temperature, and “11” indicating a hot stream). In another illustrative example, one or more bits of the stream identifier field **340** may be utilized for specifying the workload type of the data stream (e.g., “1” indicating the “log data” workload type and “0” indicating “user data” workload type). In yet another illustrative example, one or more bits of the stream identifier field **340** may be utilized for specifying the storage media type of memory devices to be utilized for storing the data items of the data stream (e.g., “000” indicating NAND/SLC, “001” indicating NAND/TLC, “010” indicating NAND/QLC, “011” indicating DRAM, “100” indicating SRAM, “101” indicating SDRAM, “110” indicating 3D cross point, etc.). Furthermore, the write stream command **300** may include various other fields which are omitted from FIG. **3** for clarity and conciseness.

The storage device controller may utilize the stream identifying information enhanced by the data attributes in order to identify the storage media type that is best suited for storing the data items of the data stream. In an illustrative example, responsive to receiving a write command specifying a data stream identifier and one or more data items to be written to a memory device, the storage device controller may parse the data stream identifier to determine one or more data attributes shared by the data items of the data stream. In certain implementations, parsing the identifier of

the data stream may involve identifying a bit string of a pre-defined size starting from a pre-defined position within the data stream identifier.

In an illustrative example, the identified data attribute may specify the anticipated retention time and/or the workload type of the data items. In one embodiment, the storage device controller may interpret the workload type attribute as indicative of the anticipated retention time, e.g., by assuming that data items associated with the “log data” attribute would have longer retention time than data items that are marked as the “user data.” The storage device controller may utilize the data attribute value to identify the storage media type that is best suited for storing the data items and the memory device that supports the identified storage media type.

In certain implementations, the storage device controller may utilize a data structure that contains a plurality of mappings of data attribute value to the storage media type and/or memory device identifier, as schematically illustrated by FIG. 4. The mapping table 400 includes a plurality of records 410A-410N, such that each record 410 specifies a data attribute value 420 and one or more identifiers 430 of memory devices that support the storage media type which is best suited for storing data items characterized by the attribute value 420. While the mapping table 400 is shown as a rectangular table, various other data structures including single or multi-dimensional arrays may be employed for storing the mapping table 400.

In certain implementations, the storage device controller may utilize one or more configurable rules, such that each rule specifies a conditional expression which is dependent upon the value of the data stream attribute and identifies a memory device to be utilized responsive to successfully evaluating the conditional expression. In an illustrative example, a configurable rule may be in the following format: IF <conditional expression=TRUE> THEN <memory device ID=device identifier>. The conditional expression may specify one or more conditions with respect to the data stream attribute values, e.g., “the data stream attribute value equals to (less than, or greater than) a pre-defined constant value,” or “the data stream attribute value falls within a specified range.” Various other formats of configurable rules may be employed by the storage device controller for identifying the memory device corresponding to the specified data stream attribute values. Responsive to identifying the memory device corresponding to the specified data stream attribute values, issue a device-level instruction causing the identified memory device to store the specified data items.

The data allocation strategy implemented by the storage device controller utilizing the mapping tables and/or configurable rules may optimize the device performance, e.g., by placing data items with a low anticipated retention time on DRAM or SRAM memory devices, which demonstrate higher performance and lower latency than NAND memory devices. In another illustrative example, the storage device controller may determine the data allocation for optimizing device endurance, e.g., by placing data items with a low anticipated retention time on DRAM or SRAM memory devices, which, unlike NAND memory devices, are not susceptible to wear caused by multiple write cycles.

In one embodiment, the storage media type that is best suited for storing the data items may be determined by the host, which may directly specify the media type by data attributes that are communicated to the storage device controller. Similarly to the above-described examples of data allocation performed by the storage device controller,

the host may determine the data allocation for optimizing device performance, endurance, and/or other operational aspects of the memory devices.

Referring again to FIG. 2, the application 210 which produces a data stream is presumably aware of its anticipated retention time and/or workload type, and thus may communicate this information to the storage driver 230 (e.g., via an Input/Output Control (IOCTL) system call). Alternatively, the data stream temperature may be determined by the storage driver 230, which may buffer the incoming data to be written to the memory devices 112A-112N, and may estimate the anticipated retention time based on the average frequency of overwrite operations requested by the application 210 with respect to one or more data items to be written to the memory devices 112A-112N. The storage driver 230 may identify, using one or more mapping tables and/or configurable rules, a storage media type which is best suited for storing the data items comprised by the data stream based on the anticipated retention time and/or workload type. Accordingly, the storage driver 230 may group the buffered data to be written to the storage device 110 into two or more data streams, and may issue stream write commands indicating the media storage type to the storage device controller 111.

Responsive to receiving a write command specifying a data attribute identifying the storage media type, the storage device controller may identify (e.g., using a mapping table) an available memory device supporting the identified storage media type, and may issue a device-level instruction causing the identified memory device to store the specified data items.

In certain implementations, the storage device controller may further utilize the stream identifying information enhanced by the data attributes in order to determine storage operation parameters (such as one or more parameters defining the data placement on the storage media) that would optimize the usage of the storage media. The storage device controller may implement one or more wear leveling methods directed to distributing the programming and erasing cycles uniformly across the media. The wear leveling methods implemented by the storage device controller may involve avoiding placing the “hot” data to the physical blocks that have experienced relatively heavy wear. The storage device controller may place the “cold” data and/or move the data that has not been modified for at least a certain period of time (e.g., a period of time exceeding a certain threshold) out of blocks that have experienced a low number of programming/erasing cycles into more heavily worn blocks. This strategy frees up the low-worn blocks for the “hot” data, while reducing the expected wear on the heavily-worn blocks.

In an illustrative example, erasing one or more data items of one data stream may require erasing one or more data items which are stored within the same or physically proximate sections of the storage media. Therefore, placing the data streams having substantially different anticipated retention time within the same or physically proximate sections of the storage media may result in excessive number of programming and erasing cycles to be performed by the controller on the storage media. Conversely, placing the data streams having similar anticipated retention time within the same or physically proximate sections of the storage media may result in reducing the number of programming and erasing cycles to be performed by the controller on the storage media. Accordingly, a storage device controller operating in one or more aspects of the present disclosure may implement a data placement strategy which is directed

to distributing the programming and erasing cycles uniformly across the media in order to maximize the endurance of the storage media. In an illustrative example, “section of the storage media” may be represented by a group of one or more memory cells such as single level cells (SLCs), multi-level cells (MLCs), or quad-level cells (QLCs) of NAND type flash memory. In another illustrative example, “section of the storage media” may be represented by groups of memory units addressable by the same signal (such as a word line or a bit line).

FIG. 5 is a flow diagram of an example method 500 of identifying memory devices based on data stream attributes, in accordance with one or more aspects of the present disclosure. The method 500 may be performed by processing logic that may include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method 500 may be performed by the storage device controller 111 of FIG. 1.

As shown in FIG. 5, at block 510, the processing logic implementing the method may receive, from a host system, a write command specifying a data item to be written to a memory device managed by the storage device controller. The write command may further specify an identifier of a data stream to which the write command belongs. In an illustrative example, the identifier of the data stream is provided by an unsigned integer value. A portion of the identifier of the data stream may encode one or more data attributes shared by the data items of the data stream. In an illustrative example, the data attribute may include a value reflecting an anticipated retention time of the data items of the data stream. In another illustrative example, the data attribute may include a value reflecting a workload type of the data items of the data stream. In another illustrative example, the data attribute may include a value specifying the storage media type to be utilized for storing the data items of the data stream, as described in more detail herein above.

At block 520, the processing logic may parse the identifier of the data stream to determine a data attribute shared by data items comprised by the data stream. In an illustrative example, parsing the identifier of the data stream may involve identifying a bit string of a pre-defined size starting from a pre-defined position within the data stream identifier, as described in more detail herein above.

At block 530, the processing logic may identify, based on the data attribute, a memory device for storing the data item. In an illustrative example, the data attribute may specify the anticipated retention time and/or the workload type of the data items. The processing logic may identify, using one or more mapping tables and/or configurable rules, a memory device supporting the storage media type which is best suited for storing the data items sharing the specified data attribute value. In another illustrative example, the data attribute may specify the storage media type which is best suited for storing the data items sharing the specified data attribute value. The processing logic may identify, using one or more mapping tables and/or configurable rules, a memory device supporting the specified storage media type, as described in more detail herein above.

At block 540, the processing logic may transmit, to the identified memory device, an instruction causing the identified memory device to store the specified data item, and the method may terminate.

FIG. 6 is a flow diagram of an example method 600 of specifying data stream attributes identifying storage media type to be utilized for storing the data items, in accordance with one or more aspects of the present disclosure. The method 600 may be performed by processing logic that may include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method 600 may be performed by the host system 120 of FIG. 1 (e.g., by the storage driver 230 of FIG. 2).

As shown in FIG. 6, at block 610, the processing logic implementing the method may receive a plurality of data items to be written to a storage device. The plurality of data items may be produced by an application running on the host system, as described in more detail herein above with references to FIG. 2.

At block 620, the processing logic may group the received data items into one or more data streams, such that the data items contained by a single data stream may share one or more attributes reflecting anticipated media usage patterns, e.g., the anticipated retention time (also referred to as the “stream temperature”) and/or the workload type. In an illustrative example, the processing logic may identify, among the plurality of data items, two or more data items sharing one or more data attributes. Based on the data attribute values, the processing logic may append the identified data items to a newly created or an existing data stream, as described in more detail herein above.

At block 630, the processing logic may identify, using one or more mapping tables and/or configurable rules, the storage media type for storing the data items sharing the specified data stream attribute values. In an illustrative example, the processing logic may utilize one or more configurable rules, such that each rule specifies a conditional expression which is dependent upon the value of the data stream attribute and identifies a memory device to be utilized responsive to successfully evaluating the conditional expression. In an illustrative example, a configurable rule may be in the following format: IF <conditional expression=TRUE> THEN <memory device ID=device identifier>. The conditional expression may specify one or more conditions with respect to the data stream attribute values, e.g., “the data stream attribute value equals to (less than, or greater than) a pre-defined constant value,” or “the data stream attribute value falls within a specified range.” The processing logic may employ various other formats of configurable rules for identifying the memory device corresponding to the specified data stream attribute values.

At block 640, the processing logic may generate a data stream identifier which includes an encoded form of the identified storage media type. In an illustrative example, the data stream identifier may be provided by an unsigned integer value, one or more bits of which may be utilized for encoding the data attributes shared by the data items of the data stream. In an illustrative example, the bit string encoding the data attributes may have a pre-defined size and may start from a pre-defined position within the data stream identifier, as described in more detail herein above.

At block 650, the processing logic may transmit, to a controller of the storage device, one or more write commands specifying the data comprised by the first data item and the second data item. Each write command may further specify the data stream identifier, the reserved part of which encodes the identified storage media type.

11

FIG. 7 is a block diagram of an example storage device controller 700, which may implement the functionality of the controller 111 of FIG. 1. As shown in FIG. 7, the controller 700 may include a host interface circuitry 714 to interface with a host system via a physical host interface 706. The host interface circuitry 714 may be employed for converting commands received from the host system into device-level instructions. The host interface circuitry 714 may be in communication with the host-memory translation circuitry 716, which may be employed for translating host addresses to memory device addresses. For example, the host-memory translation circuitry 716 may convert logical block addresses (LBAs) specified by host system read or write operations to commands directed to non-volatile memory units identified by logical unit numbers (LUNs) 750. The host-memory translation circuitry 716 may include error detection/correction circuitry, such as exclusive or (XOR) circuitry that may calculate parity information based on information received from the host interface circuitry 714.

The memory management circuitry 718 may be coupled to the host-memory translation circuitry 716 and the switch 720. The memory management circuitry 718 may control various memory management operations including, but not limited to, initialization, wear leveling, garbage collection, reclamation, and/or error detection/correction. The memory management circuitry 718 may include block management circuitry 740 which may be employed for retrieving data from the volatile memory 717 and/or non-volatile memory identified by LUNs 750. For example, the block management circuitry 740 may retrieve information such as identifications of valid data blocks, erase counts, and/or other status information of the LUNs 750. The memory management circuitry 718 may further include data allocation component 115 that may be employed to identify memory devices for storing the incoming data and/or allocate the incoming data to particular locations on the identified memory devices. It should be noted that the component designation is of a purely functional nature, i.e., the functions of the data allocation component may be implemented by one or more hardware components and/or firmware modules of the controller 700, such as the processor 728, which may be employed for implementing at least some of the above-referenced memory management operations.

The switch 720 may be coupled to the host-memory translation circuitry 716, the memory management circuitry 718, the non-volatile memory control circuitry 722, and/or the volatile memory control circuitry 724. The switch 720 may include and/or be coupled to a number of buffers. For example, the switch 720 may include internal static random access memory (SRAM) buffers (ISBs) 725. The switch may be coupled to DRAM buffers 727 that are included in the volatile memory 717. In some embodiments, the switch 720 may provide an interface between various components of the controller 700.

The non-volatile memory control circuitry 722 may store, in one of the buffers (e.g., the ISBs 725 or the buffer 727), information corresponding to a received read command. Furthermore, the non-volatile memory control circuitry 722 may retrieve the information from one of the buffers and write the information to a logical unit of the non-volatile memory identified by a LUN 750. The logical units identified by LUNs 750 may be coupled to the non-volatile memory control circuitry 722 by a number of channels. In some embodiments, the number of channels may be controlled collectively by the non-volatile memory control circuitry 722. In some embodiments, each memory channel

12

may be coupled to a discrete channel control circuit 748. A particular channel control circuit 748 may control and be coupled to more than one memory unit 750 by a single channel.

The non-volatile memory control circuitry 722 may include a channel request queue (CRQ) 747 that is coupled to each of the channel control circuits 748. Furthermore, each channel control circuit 748 may include a memory unit request queue (RQ) 744 that is coupled to multiple memory unit command queues (CQs) 746. The CRQ 747 may be configured to store commands (e.g., write requests or read requests) shared between channels, the RQ 744 may be configured to store commands between the memory units 750 on a particular channel, and the CQ 746 may be configured to queue a current command and a next command to be executed subsequent to the current command.

The CRQ 747 may be configured to receive a command from the switch 720 and relay the command to one of the RQs 744 (e.g., the RQ 744 associated with the channel that is associated with the particular logical unit identified by the LUN 750 for which the command is targeted). The RQ 744 may be configured to relay a first number of commands for a particular memory unit 750 to the CQ 746 that is associated with the particular logical unit identified by the LUN 750 in an order that the first number of commands were received by the RQ 744. A command pipeline may be structured such that commands to the logical unit move in a particular order (e.g., in the order that they were received by the RQ 744). The RQ 744 may be configured to queue a command for a particular logical unit in response to the CQ 746 associated with the particular logical unit being full and the CRQ 747 may be configured to queue a command for a particular RQ 744 in response to the particular RQ 744 being full.

The RQ 744 may relay a number of commands for different logical units identified by LUNs 750 to the CQs 746 that are associated with the logical units in an order according to the status of the logical units. For example, the logical unit status may be a ready/busy status. The command pipeline is structured such that the commands between different logical units may move out of order (e.g., in an order that is different from the order in which they were received by the RQ 744 according to what is efficient for overall memory operation at the time). For example, the RQ 744 may be configured to relay a first one of the second number of commands to a first CQ 746 before relaying a second command from the second number of commands to a second CQ 746 in response to the status of the different logical unit associated with the second CQ 746 being busy, where the first command is received later in time than the second command. The RQ 744 may be configured to relay the second command to the second CQ 746 in response to the status of the logical unit associated with the second CQ 746 being ready (e.g., subsequent to relaying the first command).

In some embodiments, the control circuits for each channel may include discrete error detection/correction circuitry 737 (e.g., error correction code (ECC) circuitry), coupled to each channel control circuit 748 and/or a number of error detection/correction circuits 737 that can be used with more than one channel. The error detection/correction circuitry 737 may be configured to apply error correction such as Bose-Chaudhuri-Hocquenghem (BCH) error correction to detect and/or correct errors associated with information stored in the logical unit identified by the LUN 750. The

13

error detection/correction circuitry **737** may be configured to provide differing error correction schemes for SLC, MLC, or QLC operations.

FIG. **8** illustrates an example computer system **800** within which a set of instructions, for causing the computer system to perform any one or more of the methodologies discussed herein, may be executed. In an illustrative example, the computer system **800** may implement the functions of the host system **120** of FIG. **1**. In alternative implementations, the computer system may be connected (e.g., networked) to other computer systems in a LAN, an intranet, an extranet, and/or the Internet. The computer system may operate in the capacity of a server or a client computer system in client-server network environment, as a peer computer system in a peer-to-peer (or distributed) network environment, or as a server or a client computer system in a cloud computing infrastructure or environment.

The computer system may be a personal computer (PC), a tablet PC, a set-top box (STB), a Personal Digital Assistant (PDA), a cellular telephone, a web appliance, a server, a network router, a switch or bridge, or any computer system capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that computer system. Further, while a single computer system is illustrated, the term “computer system” shall also be taken to include any collection of computer systems that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein.

The example computer system **800** includes a processing device **802**, a main memory **804** (e.g., read-only memory (ROM), flash memory, dynamic random access memory (DRAM)), a static memory **806** (e.g., flash memory, static random access memory (SRAM), etc.), and a data storage device **818**, which communicate with each other via a bus **830**. In an illustrative example, the data storage device **818** may implement the functions of the storage device **110** of FIG. **1**.

Processing device **802** represents one or more general-purpose processing devices such as a microprocessor, a central processing unit, or the like. More particularly, the processing device may be complex instruction set computing (CISC) microprocessor, reduced instruction set computing (RISC) microprocessor, very long instruction word (VLIW) microprocessor, or processor implementing other instruction sets, or processors implementing a combination of instruction sets. Processing device **802** may also be one or more special-purpose processing devices such as an application specific integrated circuit (ASIC), a field programmable gate array (FPGA), a digital signal processor (DSP), network processor, or the like. The processing device **802** is configured to execute instructions **826** for performing the operations and methods described herein.

The computer system **800** may further include a network interface device **808** to communicate over the network **820**. The computer system **800** also may include a video display unit **810** (e.g., a liquid crystal display (LCD) or a cathode ray tube (CRT)), an alphanumeric input device **812** (e.g., a keyboard), a cursor control device **814** (e.g., a mouse), a graphics processing unit **822**, a signal generation device **816** (e.g., a speaker), graphics processing unit **822**, video processing unit **828**, and audio processing unit **832**.

The data storage device **818** may include computer-readable storage medium **824** on which is stored one or more sets of instructions or software **826** embodying any one or more of the operations and methods described herein. The instructions **826** may also reside, completely or at least

14

partially, within the main memory **804** and/or within the processing device **802** during execution thereof by the computer system **800**, the main memory **804** and the processing device **802** also constituting computer-readable storage media. The computer-readable storage medium **824**, data storage device **818**, and/or main memory **804** may correspond to the storage device **110** of FIG. **1**.

In one implementation, the instructions **826** include instructions to implement functionality corresponding to a data allocation component (e.g., data allocation component **115** of FIG. **1**). While the computer-readable storage medium **824** is shown in an example implementation to be a single medium, the term “computer-readable storage medium” should be taken to include a single medium or multiple media (e.g., a centralized or distributed database, and/or associated caches and servers) that store the one or more sets of instructions. The term “computer-readable storage medium” shall also be taken to include any medium that is capable of storing or encoding a set of instructions for execution by the computer and that cause the computer to perform any one or more of the methodologies of the present disclosure. The term “computer-readable storage medium” shall accordingly be taken to include, but not be limited to, solid-state memories, optical media and magnetic media.

Some portions of the preceding detailed descriptions have been presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the ways used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of operations leading to a desired result. The operations are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical or magnetic signals capable of being stored, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. Unless specifically stated otherwise as apparent from the above discussion, it is appreciated that throughout the description, discussions utilizing terms such as “receiving” or “determining” or “transmitting” or “reflecting” or “specifying” or “identifying” or “providing” or the like, refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system’s registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage devices.

The present disclosure also relates to an apparatus for performing the operations herein. This apparatus may be specially constructed for the intended purposes, or it may comprise a general purpose computer selectively activated or reconfigured by a computer program stored in the computer. Such a computer program may be stored in a computer readable storage medium, such as, but not limited to, any type of disk including floppy disks, optical disks, CD-ROMs, and magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs,

15

EEPROMs, magnetic or optical cards, or any type of media suitable for storing electronic instructions, each coupled to a computer system bus.

The algorithms and displays presented herein are not inherently related to any particular computer or other apparatus. Various general purpose systems may be used with programs in accordance with the teachings herein, or it may prove convenient to construct a more specialized apparatus to perform the method. The structure for a variety of these systems will appear as set forth in the description below. In addition, the present disclosure is not described with reference to any particular programming language. It will be appreciated that a variety of programming languages may be used to implement the teachings of the disclosure as described herein.

The present disclosure may be provided as a computer program product, or software, that may include a computer-readable medium having stored thereon instructions, which may be used to program a computer system (or other electronic devices) to perform a process according to the present disclosure. A computer-readable medium includes any mechanism for storing information in a form readable by a machine (e.g., a computer system). For example, a computer-readable (e.g., computer-readable) medium includes a read only memory ("ROM"), random access memory ("RAM"), magnetic disk storage media, optical storage media, flash memory devices, etc.

In the foregoing specification, implementations of the disclosure have been described with reference to specific example implementations thereof. It will be evident that various modifications may be made thereto without departing from the broader spirit and scope of implementations of the disclosure as set forth in the following claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense.

What is claimed is:

1. A system, comprising:
a plurality of memory devices;
a processing device operatively coupled to the memory devices, the processing device to:
receive a write command specifying a data item and an identifier of a data stream comprising the data item;
determine, by parsing the identifier of the data stream, a data stream attribute shared by data items comprised by the data stream;
apply a configurable rule that, responsive to evaluating a logical expression dependent upon the data stream attribute, identifies, among the plurality of memory devices, a memory device corresponding to the data stream attribute, wherein evaluating the logical expression further comprises: comparing the data stream attribute to each of a plurality of constant values to determine a corresponding memory device type from a group comprising: Not-And (NAND) flash memory for data stream attributes matching a first logical condition comprising a first constant value, Dynamic Random Access Memory (DRAM) for data stream attributes matching a second logical condition comprising a second constant value, and Static Random Access Memory (SRAM) for data stream attributes matching a third logical condition comprising a third constant value; and transmit, to the memory device, an instruction specifying the data item.
2. The system of claim 1, wherein the data stream attribute specifies a storage media type to be utilized for storing data items comprised by the data stream.

16

3. The system of claim 2, wherein the storage media type is represented by one of: negative-and (NAND) flash memory, dynamic random access memory (DRAM), static random access memory (SRAM), or 3D cross point memory.

4. The system of claim 1, wherein the data stream attribute further specifies a workload type of data items comprised by the data stream.

5. The system of claim 1, wherein identifying the memory device further comprises:

mapping, using a memory data structure, the data stream attribute to an identifier of the memory device.

6. The system of claim 1, wherein the data stream attribute reflects a media usage pattern of the data items comprised by the data stream.

7. The system of claim 1, wherein identifying the memory device further comprises:

choosing a low latency memory device for the data stream responsive to determining that the data stream attribute specifies a low anticipated retention time.

8. A method, comprising:

receiving, by a processing device, a write command specifying a data item and an identifier of a data stream comprising the data item;

determining, by parsing the identifier of the data stream, a data stream attribute shared by data items comprised by the data stream;

applying a configurable rule that, responsive to evaluating a logical expression dependent upon the data stream attribute, identifies, among a plurality of memory devices, a memory device corresponding to the data stream attribute, wherein evaluating the logical expression comprises: comparing the data stream attribute to each of a plurality of constant values to determine a corresponding memory device type from a group comprising: Not-And (NAND) flash memory for data stream attributes matching a first logical condition comprising a first constant value, Dynamic Random Access Memory (DRAM) for data stream attributes matching a second logical condition comprising a second constant value, and Static Random Access Memory (SRAM) for data stream attributes matching a third logical condition comprising a third constant value; and transmitting, to the memory device, an instruction specifying the data item.

9. The method of claim 8, wherein the data stream attribute specifies a storage media type to be utilized for storing data items comprised by the data stream.

10. The method of claim 8, wherein the data stream attribute further specifies a workload type of data items comprised by the data stream.

11. The method of claim 8, wherein identifying the memory device further comprises:

mapping, using a memory data structure, the data stream attribute to an identifier of the memory device.

12. The method of claim 8, wherein the data stream attribute reflects a media usage pattern of the data items comprised by the data stream.

13. The method of claim 8, wherein identifying the memory device further comprises:

choosing a low latency memory device for the data stream responsive to determining that the data stream attribute specifies a low anticipated retention time.

14. A non-transitory computer-readable storage medium comprising executable instructions that, when executed by a processing device, cause the processing device to:
receive a write command specifying a data item and an identifier of a data stream comprising the data item;

17

determine, by parsing the identifier of the data stream, a data stream attribute shared by data items comprised by the data stream;

apply a configurable rule that, responsive to evaluating a logical expression dependent upon the data stream attribute, identifies, among a plurality of memory devices, a memory device corresponding to the data stream attribute, wherein evaluating the logical expression comprises: comparing the data stream attribute to each of a plurality of constant values to determine a corresponding memory device type from a group comprising: Not-And (NAND) flash memory for data stream attributes matching a first logical condition comprising a first constant value, Dynamic Random Access Memory (DRAM) for data stream attributes matching a second logical condition comprising a second constant value, and Static Random Access Memory

18

(SRAM) for data stream attributes matching a third logical condition comprising a third constant value; and transmit, to the memory device, an instruction specifying the data item.

15. The non-transitory computer-readable storage medium of claim 14, wherein the data stream attribute specifies a storage media type to be utilized for storing data items comprised by the data stream.

16. The non-transitory computer-readable storage medium of claim 14, wherein the data stream attribute further specifies a workload type of data items comprised by the data stream.

17. The non-transitory computer-readable storage medium of claim 14, wherein identifying the memory device further comprises:

mapping, using a memory data structure, the data stream attribute to an identifier of the memory device.

* * * * *